VRBC: A Verifiable Redactable Blockchain With Efficient Query and Integrity Auditing

Guohua Tian, Jianghong Wei, Mirosław Kutyłowski, Willy Susilo, *Fellow, IEEE*, Xinyi Huang, and Xiaofeng Chen, *Senior Member, IEEE*

Abstract—Driven by various legal obligations and service requirements, the redactable blockchain was introduced to balance the modifiability and immutability of blockchain technology. However, such a blockchain inevitably generates one or even more acceptable versions for the same block data, enabling malicious full nodes to deceive light/new nodes with old data and even disrupt the consistency of the blockchain ledger. In this paper, we introduce the concept of verifiable redactable blockchain (VRBC) to provide efficient validity verification for on-chain data. To this end, we design a novel authentication data structure, called blockchain authentication tree (BAT), which utilizes a chameleon hash function and aggregatable vector commitment to bind continuously-appended blocks. Based on this, we propose an efficient VRBC scheme supporting integrity auditing, which not only allows light nodes to query and validate on-chain data, but also enables new nodes to check the integrity of the blockchain ledger before synchronizing it, effectively avoiding resource waste and security risks caused by invalid queries and ledger synchronization. Furthermore, we introduce some optimized strategies to improve the performance of the proposed scheme and extend it to transaction-level and permissionless VRBC, respectively. Finally, we demonstrate the practicability of the proposed scheme through detailed security analysis and visual performance evaluation.

Index Terms—Blockchain authentication tree, Chameleon hash function, Integrity auditing, Verifiable redactable blockchain

1 Introduction

As a breakthrough computing paradigm, the blockchain technology [1], [2], [3] has attracted a lot of attention in various fields, such as insurance [4], supply chain [5], [6], and data outsourcing [7], [8], [9]. Benefiting from P2P network and consensus mechanisms, blockchain exhibits some compelling properties, especially decentralization and immutability, substantially improving the ledger's security.

- *Guohua Tian is with the State Key Laboratory of Integrated Service Networks (ISN), Xidian University, Xi'an, Shaanxi 710071, China. E-mail: gh_tian0621@163.com.*
- *Jianghong Wei is with the State Key Laboratory of Mathematical Engineering and Advanced Computing, Zhengzhou, Henan 450002, China. E-mail: jianghong.wei.xxgc@gmail.com.*
- *Mirosław Kutyłowski is with the Wrocław University of Science and Technology, 50-370 Wrocław, Poland. E-mail: miroslaw.kutylowski@pwr.edu.pl.*
- *Willy Susilo is with the Institute of Cybersecurity and Cryptology, University of Wollongong, Wollongong, NSW 2522, Australia. E-mail: wsusilo@uow.edu.au.*
- *Xinyi Huang is with the Artificial Intelligence Thrust, Information Hub, Hong Kong University of Science and Technology (Guangzhou), Guangzhou, Guangdong Province 511455, China. E-mail: xinyi@ust.hk.*
- *Xiaofeng Chen is with the State Key Laboratory of Integrated Service Networks (ISN), Xidian University, Xi'an, Shaanxi 710071, China, and also with the College of Computer and Cyber Security, Fujian Normal University, Fuzhou, Fujian 350117, China. E-mail: xfchen@xidian.edu.cn.*

Manuscript received 17 August 2022; revised 17 October 2022; accepted 27 November 2022. Date of publication 20 December 2022; date of current version 9 June 2023.
This work was supported in part by the National Nature Science Foundation of China under Grants 61960206014, 62032012, 62121001, and 62172434, in part by the Key Research and Development Program of Shaanxi under Grant 2020ZDLGY08-03.
(Corresponding author: Xiaofeng Chen.)
Recommended for acceptance by D. He.
Digital Object Identifier no. 10.1109/TC.2022.3230900

It turns out that immutability is a double-edged sword that creates limitations for the development of blockchain. Malicious users may contaminate the blockchain by posting some illegal/improper information (e.g., violent crimes [10], [11]), or launch attacks using potential vulnerabilities (e.g., DAO [12]). Moreover, the continuous shift towards privacy protection (e.g., GDPR [13]) has intensified the conflict between the "right to be forgotten" [14] and blockchain immutability. As a result, future blockchain technologies will need to trade off between modifiability and immutability.

To this end, Ateniese et al. [15] proposed the concept of redactable blockchain to achieve controlled redaction on the blockchain. By adopting a chameleon hash (CH) function [16], the redactor can use trapdoor key to generate a collision for the target block, and elegantly redact it without breaking the hash link between adjacent blocks. Based on this, several schemes have been proposed to achieve fine-grained privilege distribution [17], [18], [19], [20], [21], accountability [18], [19], [22], and supervised self-management [23].

Once the blockchain becomes redactable, there may be multiple acceptable versions of the same block/transaction. While full nodes (FNs) can verify and update each redacted data promptly, light nodes (LNs) that only maintain block headers cannot effectively validate the block data obtained from FNs. That is, malicious FNs can deceive LNs, especially new nodes that want to synchronize his ledger. In this case, if and only if all redaction records have been recorded on the blockchain, light/new nodes can backtrack to the genesis block to verify the correctness and validity of blockchain ledger. Distinctly, this is infeasible for resource-constrained LNs. Taking into account the necessity of efficient data query and auditing, existing primitives, such as Provable Data Possession (PDP) [24], Proof of Retrieval (POR)

TABLE 1
Comparisons of Redactable Blockchain

Schemes	Main idea	Setting	Authorization	Redactor	Efficient query	Auditing
AMVA17 [15]	CH	Permissioned/less	CA/Committee	CA/Committee	×	×
PDS17 [31]	Multi-version	Permissioned	CA	CA	×	×
DSSS19 [17]	PCH	Permissioned/less	CA/Users	Authorized users	×	×
DMT19 [32]	Voting	Permissionless	N/A	All miners	×	×
TLL+20 [18]	PCHBA	Permissioned	CA	Authorized users	×	×
PVN21 [19]	RCHET	Permissioned	CA	Authorized users	×	×
XNMX+21 [20]	RPCH	Permissioned	CA	Authorized users	×	×
XNMH+21 [22]	CH+Signature	Permissioned	CA	Authorized users	×	×
JSZ+21 [23]	sCHRS	Permissioned	CA	CA + All users	×	×
MXNH+22 [21]	DPCH	Permissioned	Multi-CA	Authorized users	×	×
Our scheme	BAT	Permissioned/less	CA/Committee	CA/Committee	✓	✓

**Efficient query*: *LN can query on-chain data and validate its correctness and validity without traversing the entire blockchain.*

[25], [26], and Verifiable Data Streaming (VDS) [27], [28], [29], [30], cannot be applied into redactable blockchains since the blockchain ledger is generated by all miners rather than a single user. Therefore, none of the existing redactable blockchains can solve this problem, which raises a significant question :

Can we design a verifiable redactable blockchain with efficient query and integrity auditing?

1.1 Our Contributions

Aiming at the above problems, we propose a verifiable redactable blockchain (VRBC) with efficient query and integrity auditing, mainly involving three contributions:

- *Blockchain authentication tree (BAT)*. We design a novel authentication data structure BAT, customized for redactable blockchain, which binds the continuously-appended blocks to provide elegant redaction and efficient validity verification for on-chain data.
- *Efficient VRBC*. Based on BAT, we propose an efficient VRBC scheme supporting integrity auditing. It enables light/new nodes to query and validate the on-chain data or check the integrity of the blockchain ledger before synchronizing it. Notably, our auditing protocol can act as a subroutine of consensus mechanism to prevent malicious miners from appending new blocks to the blockchain.
- *Optimized and extended VRBCs*. Taking advantage of the structural characteristics of BAT, we propose optimized auditing and delayed redaction strategy to improve the performance of the proposed VRBC scheme. Also, we extend it to a transaction-level VRBC and a permissionless VRBC, respectively.

1.2 Related Work

As shown in Table 1, the current mainstream redactable blockchains are built upon various CHs or their variants. Inspired by the work of Ateniese et al. [15], Derler et al. [17] formalized the concept of Policy-based CH (PCH) and presented a fine-grained redactable blockchain. By using the chameleon hash with ephemeral trapdoor (CHET) [33] to compute the hash value for each transaction, they achieve transaction-level redaction on the blockchain. Meanwhile, the ephemeral trapdoor will be distributed through ciphertext-policy attribute-based encryption (CP-ABE) [34], enabling users whose attributes issued by the central authority (CA) satisfy the corresponding access policy to redact target transactions. Furthermore, Xu et al. [20] proposed a revocable PCH (RPCH). It realizes more flexible privilege management in a redactable blockchain by replacing the PCH's standard CP-ABE with a revocable one. After that, Ma et al. [21] proposed a novel decentralized PCH (DPCH) that enables a reliable redactable blockchain by introducing multiple authorities to issue redaction privileges. Furthermore, Puddu et al. [31] proposed a novel redactable blockchain, named μchain. Unlike CH-based solutions, a μchain stores multiple versions of encrypted transactions and employs CA to control the access view of miners by releasing the decryption keys of the currently active transactions. Although μchain is an effective solution for patching smart contracts (e.g., DAO), it has to store all redundant transactions permanently, and cannot realize adaptive redaction due to the limited number of the pre-set options.

To prevent redactors from abusing redaction privilege, Tian et al. [18] proposed a new PCH that supports black-box accountability (PCHBA). Namely, it enables CA to link malicious redactions to its redactor and allows any user to identify them by interacting with the black-box device. Similarly, Panwar et al. [19] presented a revocable and traceable redactable blockchain, in which a revocable CHET (RCHET) integrated with the group signature is utilized to trace malicious redactors. Moreover, Jia et al. [23] presented a novel variant of CH, named stateful chameleon hash with revocable subkey (sCHRS). It allows users to redact the private data of their transactions with their sCHRS subkey. For supervising malicious redactions, a semi-trusted CA is employed to revoke malicious users once they redact the data beyond their privileges. Besides, Xu et al. [22] presented an epoch-based K-time redactable blockchain, in which each redactor needs to pay a time-locked deposit to obtain K-time redaction privileges from CA. Once a redactor implements a malicious redaction on the blockchain during the current epoch, his deposit will be forfeited.

Considering the redactable blockchain in a permissionless setting, Ateniese et al. [15] introduced a decentralized architecture for CH-based redactable blockchains, which shares a trapdoor key among some fixed users who possess a lot of system resources (e.g., the top 7 mining pools), and allows them to redact blocks via a secure multi-party computation (MPC) protocol. Similarly, Derler et al. [17] extended their

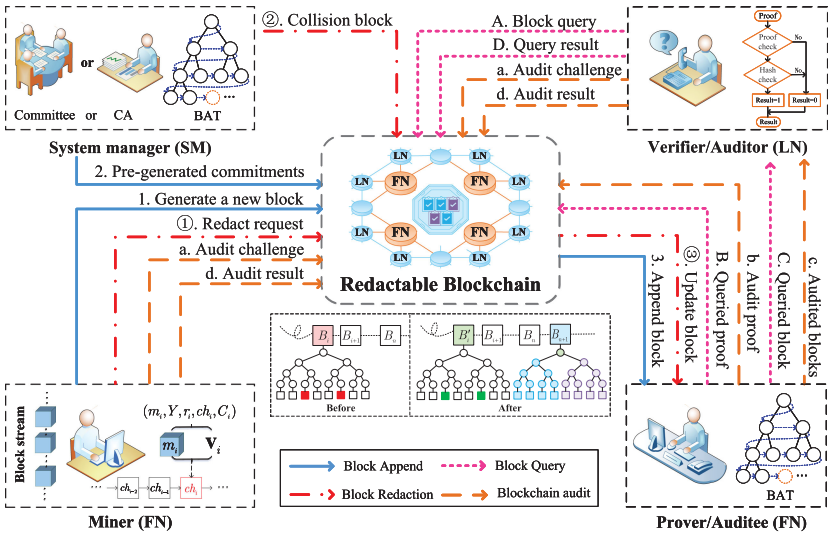


Fig. 1. The system model and workflow of the proposed VRBC scheme.

scheme to a permissionless setting, where each user acts as its own attribute authority and issues redaction privilege to others. Furthermore, Deuber et al. [32] proposed a redactable blockchain based on a voting mechanism [35], in which every two adjacent blocks are linked with two hash chains and the target block will be replaced with a new one that has obtained enough votes from the following blocks. In this way, redactions only change a particular hash chain of blocks, while the other one never changes.

2 Problem Formulation

In this section, we introduce the system model, definitions, and threat model of the proposed scheme.

2.1 System Model

As shown in Fig. 1, the underlying architecture of the proposed scheme involves the following four types of entities.

- *System manager (SM)* is a fully trusted initiator, such as a CA or committee, responsible for initializing the VRBC system. Besides, **SM** pre-generates the commitment for each block, which enables blockchain nodes to build a BAT based on the redactable blockchain. Also, **SM** can redact on-chain data according to legal obligations or user requests.
- *Blockchain miner (BM)* is a full node (FN) holding the entire blockchain ledger, and responsible for appending new blocks to the blockchain. In addition, **BM** can monitor malicious on-chain data and issue redact requests on the blockchain, or check the correctness of audit proofs published on the blockchain.
- *Verifier/Auditor (VA)* is a light/new node (LN), such as a resource-constrained user that can query and validate the on-chain data, or a new user that can check the integrity of the blockchain ledger held by FNs before synchronizing it.
- *Prover/Auditee (PA)* is a full node holding the entire blockchain ledger, and responsible for providing verifiable data query or ledger synchronization services.

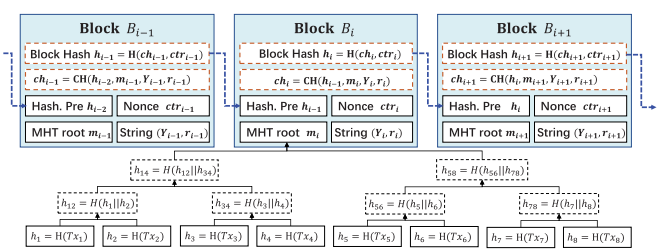


Fig. 2. The redactable blockchain structure of VRBC.

2.2 Definitions

Inspired by [15], we give the definition of VRBC based on the chameleon hash $\mathcal{CH} = \{\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{CH}, \mathsf{Col}, \mathsf{Verify}\}$ in [36] and the aggregatable vector commitment $\mathcal{AVC} = \{\mathsf{Setup}, \mathsf{Commit}, \mathsf{Update}, \mathsf{Open}, \mathsf{Aggreagte}, \mathsf{Verify}\}$ in [37].

- *Redactable blockchain*: The VRBC blockchain, as shown in Fig. 2, consists of a sequence of redactable blocks $B_i = (h_{i-1}, ch_i, m_i, Y_i, r_i, ctr_i)$, in which $h_{i-1}, r_i \in \mathbb{Z}_p^*, ch_i, Y_i \in \mathbb{G}, m_i \in \{0,1\}^\lambda$, and $ctr_i \in \mathbb{N}$. Take appending block B_i as an example, **BM** uses the root value m_i of the Merkle hash tree (MHT) built upon transactions $\{Tx_1, ..., Tx_8\}$ to generate chameleon hash value for the new block B_i:

$$ch_i = \mathsf{CH}(h_{i-1}, m_i, Y_i, r_i) \tag{1}$$

  where h_{i-1} is the hash value of the previous block, and (Y_i, r_i) is the verification string of (m_i, ch_i) in CH function [36]. Then, the block hash h_i is generated through a standard hash function $\mathsf{H}(ch_i, ctr_i)$, where ctr_i is the Nonce value of the block B_i.
- *Verifiable BAT*: VRBC adopts vector commitment to bind the above blockchain as a q-ary spanning-tree BAT. As depicted in Fig. 3[1], take appending block B_4 to a binary BAT as an example, the commitment value of node n_4 is generated in advance when its

1. Generating a BAT mainly involves metadata in each block header, for brevity, we omit all block bodies (transactions) in this figure.

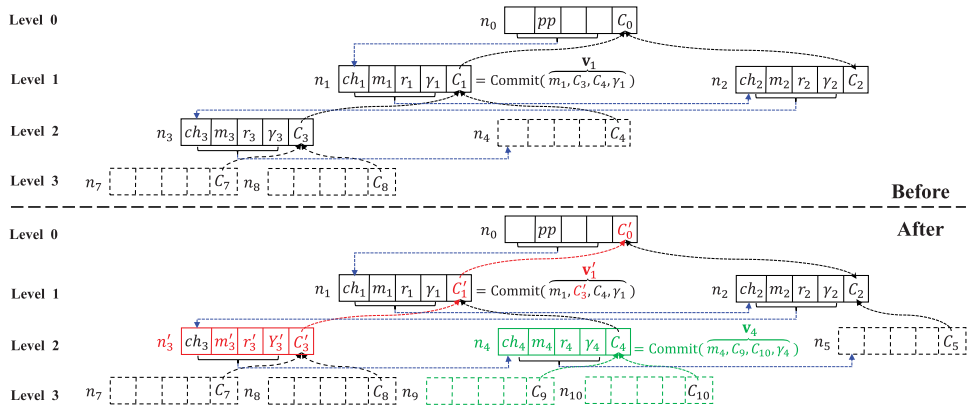


Fig. 3. The process of appending a new block B_4 to node n_4 and redacting block B_3 with B_3' in binary BAT (i.e., $(q = 2)$).

parent node B_1 is appended to BAT. Now, **SM** pre-generates the commitments of B_4's child nodes and calculates a verification value γ_4, such that:

$$C_4 = \mathsf{Commit}(m_4, C_9, C_{10}; \gamma_4) \tag{2}$$

where Commit is an improved algorithm of Point-Proofs scheme [37] obtained by introducing a verification value, which enables **SM** to append a new block into BAT without updating the entire BAT.

For better readability, we define some auxiliary functions for q-ary BAT: On input node index i and parameter q,

- $\mathsf{Level}(i, q) = \lfloor \log_q((q-1)(i+1)+1) - 1 \rfloor$: it outputs the level l of the i-th node n_i
- $\mathsf{Parent}(i, q) = \lfloor \frac{i-1}{q} \rfloor$: it outputs the index of parent node of the node n_i;
- $\#\mathsf{Child}(i, q) = ((i-1) \bmod q) + 2$: it outputs the position of node n_i in its parent node commitment;
- $\mathsf{Path}(i, q) = \mathbf{P}_i$: it outputs the index set of path nodes from n_i to the root node: $\mathbf{P}_i = (p_0, ..., p_l)$, where $p_0 = 0, p_l = i$ and $p_{j-1} = \mathsf{Parent}(p_j, q)$, for $j \in [1, l]$;
- $\mathsf{Fam}(i, q) = \mathbf{A}_i$: it outputs $\mathbf{A}_i = (a_1, a_2, ..., a_{q+1})$, the index vector of the node n_i and its children in the tree, called later a *family index vector*. Namely, $a_1 = i$, and $a_j = q \cdot i + j - 1$, for $1 < j \leq q + 1$.

Essentially, VRBC is an auditable VDS protocol customized for redactable blockchain, involving four algorithms (*Setup*, *KeyGen*, *Bind*, *Redact*) and two protocols (*Query*, *Audit*).

1) **Setup**$(1^\lambda, 1^N) \rightarrow pp$: On input security parameters $(1^\lambda, 1^N)$, this algorithm outputs public parameters pp.
2) **KeyGen**$(1^\lambda, 1^N) \rightarrow (TK, HK)$: On input $(1^\lambda, 1^N)$, the key generation algorithm outputs a secret trapdoor key TK and public hash key HK.
3) **Bind**$(h_{i-1}, m_i, HK, TK) \rightarrow (ch_i, r_i, \{C_j\}_{j \in \mathbf{A}_i}, \gamma_i)$: On input m_i, h_{i-1}, TK and HK, the bind algorithm outputs a CH value ch_i and child node commitments $\{C_j\}_{j \in \mathbf{A}_i}$:
   - $\mathsf{CH}(m_i, h_{i-1}, HK) \rightarrow (ch_i, r_i)$: it returns a hash value ch_i and verification string (Y, r_i).
   - $\mathsf{Commit}(m_i, TK) \rightarrow (\{C_j\}_{j \in \mathbf{A}_i}, \gamma_i)$: it returns commitments $\{C_j\}_{j \in \mathbf{A}_i}$ and verification value γ_i.
4) **Redact**$(m_s', m_s, HK, TK) \rightarrow (Y_s', r_s', \{C_i'\}_{i \in \mathbf{P}_s})$: On input a new data m_s' and old data m_s, the redaction algorithm outputs the updated verification string (Y_s', r_s') and the related path node commitments $\{C_i'\}_{i \in \mathbf{P}_s}$:
   - $\mathsf{Col}(TK, HK, m_s', h_{s-1}) \rightarrow (Y_s', r_s')$: it returns a collision (m_s', Y_s', r_s') for (m_s, Y_s, r_s).
   - $\mathsf{Update}(m_s, m_s', \{C_i\}_{i \in \mathbf{P}_s}) \rightarrow \{C_i'\}_{i \in \mathbf{P}_s}$: it returns the updated commitments $\{C_i'\}_{i \in \mathbf{P}_s}$.
5) **Query**: This interactive protocol can query and verify the correctness and validity of on-chain data, as follows:
   - $\mathsf{Chal}(pp) \rightarrow s$: takes pp as input, this algorithm outputs the index s of the queried block.
   - $\mathsf{Prove}(s, BAT) \rightarrow \pi_s$: based on s, this procedure uses BAT to run Open and Aggregate algorithms, and returns the proof π_s for the queried block m_s.
   - $\mathsf{Verify}(m_s, \pi_s) \rightarrow 0/1$: based on the proof π_s and the queried block m_s, this procedure outputs 1 if the proof π_s is valid. Otherwise, it outputs 0.
6) **Audit**: This interactive protocol can check the integrity of the blockchain ledger, as follows:
   - $\mathsf{Chal}(pp) \rightarrow Chal$: takes pp as input, this algorithm outputs a challenge $Chal$.
   - $\mathsf{Prove}(Chal, BAT) \rightarrow \hat{\pi}$: based on $Chal$, this procedure uses BAT to run Open and Aggregate algorithms, and returns an integrity proof $\hat{\pi}$.
   - $\mathsf{Verify}(\hat{\pi}, \{m_i\}_{i \in Chal}) \rightarrow 0/1$: based on the proof $\hat{\pi}$ and the challenged data $\{m_i\}$, this procedure outputs 1 if $\hat{\pi}$ is valid. Otherwise, it outputs 0.

2.3 Threat Model

For better understanding, we specify a threat model for the proposed VRBC scheme, involving three security goals of correctness, soundness, and controlled redaction.

- **Correctness**: In the block query or blockchain auditing phase, the Verify procedure outputs 1 for a given query/audit challenge $Chal$, if the system parameters have been generated by **Setup**, the data structure has been generated by **KeyGen**, **Bind** and **Redact**, and the proof has been generated by Prove.
- **Soundness**: If a prover can convince a verifier that the challenged block data is intact and valid through

completing a query or audit challenge, it actually stores those data. Specifically, the soundness experiment between a challenger $\mathcal{C}$ and a PPT adversary $\mathcal{A}$ involves the following phases:

- *Initialization*: The challenger $\mathcal{C}$ first runs **Setup** and **KeyGen** to generate public parameter pp. Then, he samples a set of data $\{m_i\}_{i\in N}$ and runs Commit to generates its commitment C. Finally, $\mathcal{C}$ sends (pp, C, N) to the adversary $\mathcal{A}$;
- *Query*: $\mathcal{A}$ picks a random index $j \in [N]$ and asks $\mathcal{C}$ to return the corresponding data $m_j \in \{m_i\}$ and its proof π_j until the number of queries reaches upper bound $q_H < N$;
- *Challenge*: $\mathcal{C}$ checks $\mathcal{A}$ with a challenge $Chal$ consisting of some data that have not been queried, $\mathcal{A}$ returns the corresponding proof π;
- *Verify*: $\mathcal{C}$ outputs 1 if π is an accepting proof to $(Chal, \{C\})$. Otherwise, outputs 0.

- *Controlled redaction*: Given a collision (m'_s, Y'_s, r'_s) for (m_s, Y_s, r_s), no adversary can extract the corresponding trapdoor key. Besides, except for the redactors who hold the trapdoor key, no user can generate a new collision (m''_s, Y''_s, r''_s) for (m_s, Y_s, r_s).

3 Verifiable Redactable Blockchain

In this section, we describe the concrete construction of the proposed VRBC scheme.

3.1 Overview

Inspired by [15], [28], we design a novel blockchain authentication tree (BAT) that enables LNs to query or audit the redactable blockchain ledger held by FNs. Specifically, **BM**s adopts a chameleon hash function CH [36] to generate a new block. If **BM** wins the current consensus process, his new block will be appended to the blockchain. Then, **SM** uses an aggregatable vector commitment to bind the new block into the q-ary BAT. Later, **SM** will follow redaction requests to generate and publish the corresponding collision block and updated BAT root commitment on the blockchain. On this basis, **VA** will be able to query the on-chain block from **PA**s and verify its correctness and validity. In addition, the new node will be able to check the integrity of the blockchain ledger before synchronizing it. Notably, all audit results will be published on the blockchain and shared among all nodes, which avoids the resource waste incurred by repeated auditing within the same period, enabling all nodes to supervise and exclude malicious **PA**s.

3.2 The Concrete Construction

3.2.1 System Setup

Given two security parameters $(1^\lambda, 1^N)$, **SM** runs **Setup** algorithm to initialize a VRBC system. First, **SM** generates three bilinear groups $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$ of prime order p to build a bilinear pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$, the generators of those groups are g_1, g_2 and $g_T = e(g_1, g_2)$, respectively. Then, he defines two pseudo-random functions $\phi_1 : \{1, 2, ..., n\} \times \mathbb{Z}_p^* \to \{1, 2, ..., n\}$ and $\phi_2 : \{1, 2, ..., n\} \times \mathbb{Z}_p^* \to \mathbb{Z}_p^*$, along with two collision-resistant hash functions $\mathsf{H}_1 : \mathbb{Z}_p^* \times \mathbb{G} \to \mathbb{Z}_p^*$ and $\mathsf{H}_2 : \{0,1\}^* \to \mathbb{Z}_p^*$. After that, he runs **KeyGen** algorithm to generate trapdoor keys $x, y, \alpha \leftarrow \mathbb{Z}_p^*$, hash key $HK = (X, Y) = (g_1^x, g_1^y)$ and parameter (column) vectors:

$$\begin{aligned} g_1^{\mathbf{a}} &= (g_1^{\alpha}, ..., g_1^{\alpha^N}) \\ g_1^{\alpha^N \mathbf{a}[-1]} &= (g_1^{\alpha^{N+2}}, ..., g_1^{\alpha^{2N}}) \\ g_2^{\mathbf{a}} &= (g_2^{\alpha}, ..., g_2^{\alpha^N}) \end{aligned} \tag{3}$$

where (column) vector $\mathbf{a} := (\alpha, \alpha^2, ..., \alpha^N)$. Furthermore, **SM** packages the public parameter set $pp = (e, p, g_1, g_2, g_T, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, \mathsf{H}_1, \mathsf{H}_2, \phi_1, \phi_2, HK, g_1^{\mathbf{a}}, g_1^{\alpha^N \mathbf{a}[-1]}, g_2^{\mathbf{a}})$ and generates the genesis block B_0, as follows.

$$\begin{cases} h_0 = \mathsf{H}_2(pp || ctr_0) \\ C_0 = g_1^{\mathbf{v}_0^\top \mathbf{a}} = g_1^{\sum_{i=1}^N v_{0,i}\alpha^i} \end{cases} \tag{4}$$

where $\mathbf{v}_0 = (h_0, C_1, ..., C_q)$ and $C_j = g_1^{\kappa_j}, \kappa_j = \mathsf{H}_2(x||j)$. Finally, **SM** broadcasts $B_0 = (h_0, ctr_0, pp, \{C_j\}_{j\in\mathbf{A}_0})$ and root node commitment C_0 on the blockchain network.

3.2.2 Block Append

To append a block to the blockchain, **BM** packages some transactions and calculates their MHT root m_i. Then, he runs **Bind** algorithm to generate a new block B_i.

- *Hash binding*: **BM** picks at random an element $r_i \leftarrow \mathbb{Z}_p^*$ and computes the chameleon hash value of B_i:

$$\begin{aligned} ch_i &= (X \cdot Y)^{\mathsf{H}_1(h_{i-1}||m_i, Y)} \cdot g_1^{r_i} \\ &= g_1^{\mathsf{H}_1(h_{i-1}||m_i, Y)(x+y)+r_i} \end{aligned} \tag{5}$$

  where $h_{i-1} = \mathsf{H}_2(ch_{i-1}, ctr_{i-1})$. Then, **BM** appends block $B_i = (h_{i-1}, ch_i, m_i, Y, r_i)$ to the blockchain when he wins the current consensus process.
- *Commitment binding*: Upon receiving B_i, **SM** executes the procedure Commit to bind B_i to q-ary BAT ($q = N - 1$). As the instance shown in Fig. 3, **SM** runs $\mathsf{Fam}(i, q)$ function to derive the family index vector $\mathbf{A}_i = (a_1, a_2, ..., a_{q+1})$. For each $j \in [2, q+1]$, **SM** computes the trapdoor $\kappa_{a_j} = \mathsf{H}_2(x||a_j)$ and its commitment $C_{a_j} = g_1^{\kappa_{a_j}}$ for the future j-th child node of node n_i. Then, **SM** obtains a family commitment vector $\mathbf{v}_i = (m_i, C_{a_2}, ..., C_{a_{q+1}})$ and calculates a verification value $\gamma_i = \kappa_i - \mathbf{v}_i^\top \mathbf{a}$, such that:

$$\begin{aligned} C_i = g_1^{\kappa_i} &= g_1^{\gamma_i + \sum_{j=1}^N v_{i,j}\alpha^j} \\ &= g_1^{\gamma_i + m_i\alpha + C_{a_2}\alpha^2 + \cdots + C_{a_N}\alpha^N} \end{aligned} \tag{6}$$

  Finally, **SM** publishes the family commitment vector $\mathbf{v}_i$ and commitment verification value γ_i of the new block B_i on the blockchain.
- Upon receiving a new block B_i along with $(\mathbf{v}_i, \gamma_i)$, all LNs check its correctness, as follows:

$$\begin{cases} C_i \stackrel{?}{=} g_1^{\gamma_i + \sum_{j=1}^N v_{i,j}\alpha^j} \\ ch_i \stackrel{?}{=} (X \cdot Y)^{\mathsf{H}_1(h_{i-1}||m_i, Y)} \cdot g_1^{r_i} \end{cases} \tag{7}$$

If equation (7) holds, they append B_i to their local blockchain ledger. That is, they insert the new node $n_i = (ch_i, m_i, r_i, \gamma_i, C_i)$ to their local copy of BAT.

Algorithm 1. Commitment Update

Input: $(s, m_s, m'_s, \{C_i\}_{i\in\mathbf{P}_s})$
Output: $(\{c'_i\}_{i\in\mathbf{P}_s})$
Compute $l \leftarrow \mathsf{Level}(s,q)$ and $b \leftarrow \mathsf{Parent}(s,q)$
Set $i \leftarrow s$
for $(h = l; h \geq 0; h--)$ **do**
  if $h == l$ **then**
    $C'_s = C_s \cdot g_1^{(m'_s - m_s)\alpha - \gamma_s}$, $\gamma_s \leftarrow 0$
  else
    $c \leftarrow \#\mathsf{Child}(i,q)$
    $C'_b = C_b \cdot g_1^{(C'_i - C_i)\alpha^c - \gamma_b}$, $\gamma_b \leftarrow 0$
    $i \leftarrow b$, $b \leftarrow \mathsf{Parent}(b,q)$
  end
end

Algorithm 2. Query Proof Generation

Input: (s, BAT)
Output: $(\pi_s, \mu_s, h_{s-1}, ch_s, m_s, Y_s, r_s, \{C_i\}_{i\in\mathbf{P}_s})$
Set $l \leftarrow \mathsf{Level}(s,q)$
Compute $\mathbf{P}_s = \mathsf{Path}(s,q) = (p_1, p_2, ..., p_l)$
Compute $t_s = \mathsf{H}_2(c, C_s, \mathbf{v}_s[c])$ where $c = 1$
Compute $\pi_s \leftarrow (C_s / g_1^{m_s\alpha})^{\alpha^{N+1-c} t_s}$
for $(i \in P_s \wedge i \neq 0)$ **do**
  $b \leftarrow \mathsf{Parent}(i,q)$
  $c \leftarrow \#\mathsf{Child}(i,q)$
  Run $\mathsf{Fam}(b,q)$ to generate $\mathbf{A}_b = (a_1, ..., a_{q+1})$
  Get $\mathbf{v}_b = (m_b, C_{a_2}, ..., C_{a_{q+1}})$
  Compute $t_b = \mathsf{H}_2(c, C_b, \mathbf{v}_b[c])$
  Generate $\pi_b = (C_b / g_1^{v_{b,c}\alpha^c})^{\alpha^{N+1-c} \cdot t_b}$
end
Compute $\pi_s \leftarrow \prod_{i\in\mathbf{P}_s} \pi_i$, $\mu_s \leftarrow \sum_{i\in\mathbf{P}_s} v_{i,c} t_i$
Get $B_s = (h_{s-1}, ch_s, m_s, Y_s, r_s)$, $\{C_i\}_{i\in\mathbf{P}_s}$ from BAT.

3.2.3 Block Redaction

When the s-th block needs to be redacted (for example, we have to replace m_s by m'_s, similar to the redaction instance shown in Fig. 3), **SM** executes the following procedures:

- **Update.Col**: **SM** computes the long-term trapdoor k_s of the block chameleon hash ch_s:

$$k_s = \mathsf{H}_1(h_{s-1} || m_s, Y_s) \cdot (x + y_s) + r_s \bmod p \tag{8}$$

  Then, **SM** generates a new ephemeral trapdoor key $y'_s = \mathsf{H}_2(x, m'_s)$ and its hash key $Y'_s = g_1^{y'_s}$. After that, SM calculates the new verification value r'_s:

$$r'_s = k_s - \mathsf{H}_1(h_{s-1} || m'_s, Y'_s) \cdot (x + y'_s) \bmod p \tag{9}$$

  In this way, **SM** generates a collision block $B'_s = (ch_s, m'_s, Y'_s, r'_s)$ for $B_s = (ch_s, m_s, Y_s, r_s)$.
- *Update.Com*: To revoke the invalid block data m_s, **SM** updates the commitments $\{C_i\}_{i\in\mathbf{P}_s}$ of the path nodes from the redacted block m'_s to the root node in sequence, as shown in *Algorithm 1*. Finally, **SM** publishes the new block (s, m'_s, Y'_s, r'_s) and the updated root commitment C''_0 on the blockchain.
- *Update.block*: After obtaining $(s, m'_s, Y'_s, r'_s, C''_0)$, all FNs run *Algorithm 1* to generate the new root commitment C'''_0. If $C'''_0 = C''_0$, they further check that:

$$ch_s \stackrel{?}{=} (X \cdot Y'_s)^{\mathsf{H}_1(h_{s-1} || m'_s, Y'_s)} \cdot g_1^{r'_s} \tag{10}$$

  If equation (10) does not hold, then FNs aborts this procedure. Otherwise, they replace the block $(ch_s, m_s, r_s, \gamma_s, C_s)$ and the commitments $\{C_i\}_{i\in\mathbf{P}_s}$ with $(ch_s, m'_s, r'_s, Y'_s, C'_s)$ and $\{C'_i\}_{i\in\mathbf{P}_s}$, respectively.

Algorithm 3. Audit Proof Generation

Input: $(Chal, BAT)$
Output: $(\hat{\pi}, \hat{\mu}, \{h_{s-1}, ch_s, m_s, Y_s, r_s, \{C_i\}_{i\in\mathbf{P}_s}\}_{s\in B})$
Set $\hat{\pi} \leftarrow 1$
Parse $Chal$ as z, f_1 and f_2
for $(i = 1; i \leq z; i++)$ **do**
  Compute index: $b_i = \phi_1(i, f_1)$
  Compute coefficient: $t_i = \phi_2(i, f_2)$
  Compute $t_{i,c} = \mathsf{H}_2(c, C_{b_i}, \mathbf{v}_{b_i}[c])$ where $c = 1$
  Compute $\pi_{b_i} = (C_{b_i} / g_1^{m_{b_i}\alpha})^{\alpha^{N+1-c} t_{i,c} t_i}$
  Get $\hat{\pi} \leftarrow \hat{\pi} \cdot \pi_{b_i}$
end
Set $B \leftarrow \{b_i\}_{i\in[z]}$
while $B \neq \{0\}$ **do**
  $s \leftarrow \max(B)$, $b \leftarrow \mathsf{Parent}(s,q)$
  Run $\mathsf{Fam}(b,q)$ to generate $\mathbf{A}_b = (a_1, ..., a_{q+1})$
  Get $\mathbf{v}_b = (m_b, C_{a_2}, ..., C_{a_{q+1}})$
  $D_b \leftarrow \mathbf{A}_b[-1] \cap B$
  $S_b \leftarrow \{\#\mathsf{Child}(d,q)\}$ where $d \in D_b$
  $t_b = \mathsf{H}_2(C_b)$
  for $c \in S_b$ **do**
    $\pi_b \leftarrow 1$
    $t_{b,c} = \mathsf{H}_2(c, C_b, \mathbf{v}_b[S_b])$
    $\pi_b \leftarrow \pi_b \cdot (C_b / g_1^{v_{b,c}\alpha^c})^{\alpha^{N+1-c} \cdot t_{b,c} t_b}$
  end
  $B \leftarrow (B / D_b) \cup \{b\}$
end
Get $\hat{\pi} \leftarrow \hat{\pi} \cdot \prod_{i\in\mathbf{P}_s, s\in B} \pi_i$, $\hat{\mu} \leftarrow \sum_{i\in\mathbf{P}_s, s\in B} \sum_{c\in S_i} v_{i,c} t_{i,c} t_i$
Get $\{h_{s-1}, ch_s, m_s, Y_s, r_s, \{C_i\}_{i\in\mathbf{P}_s}\}_{s\in B}$ from BAT.

3.2.4 Block Query

To query the s-th block data m_s, **VA** can perform an interactive protocol **Query** with a full node **PA**, as follows[2]:

- **Chal**: **VA** sends a request to **PA** and asks for the s-the block data along with its validity proof.
- **Proof**: **PA** runs *Algorithm 2* to generate the query proof (π_s, μ_s), which will be published on the blockchain. Meanwhile, **PA** sends the verification metadata $(h_{s-1}, ch_s, m_s, Y_s, r_s, \{C_i\}_{i\in\mathbf{P}_s})$ of the queried block to **VA** via a secure off-chain channel.
- **Verify**: After getting the queried block $B_s = (h_{s-1}, ch_s, m_s, Y_s, r_s, \{C_i\}_{i\in\mathbf{P}_s})$ and proof (π_s, μ_s), if

2. In practice, a light node can obtain the latest root commitment C''_0 from other full nodes.

$C'_0 = C_0$, **VA** checks the validity of queried block:

$$\prod_{i \in \mathbf{P}_s} e(C_i, g_2^{\alpha^{N+1-c} t_i}) \stackrel{?}{=} e(\pi_s, g_2) \cdot g_T^{\alpha^{N+1} \mu_s} \tag{11}$$

where $c = \#\mathsf{Child}(j,q)$ and $i = \mathsf{Parent}(j,q), (i,j \in \mathbf{P}_s)$. If equation (11) does not hold, which means that the block data m_s is invalid, **VA** discards it. Otherwise, he further checks the correctness of the queried block via validating its chameleon hash value:

$$ch_s \stackrel{?}{=} (X \cdot Y_s)^{\mathsf{H}_1(h_{s-1} || m_s, Y_s)} \cdot g_1^{r_s} \tag{12}$$

If both equations (11) and (12) are valid, **VA** remains B_s locally. Otherwise, he discards it.

3.2.5 Blockchain Auditing

To check the integrity of the blockchain ledger held by **PA**, the auditor **VA** can execute an **Audit** protocol, as follows.

- **Chal**: **VA** picks three random elements $z \in [n]$ and $f_1, f_2 \in \mathbb{Z}_p^*$. Then, he issues the audit challenge $Chal = (z, f_1, f_2)$ on the blockchain.
- **Proof**: According to the challenge $Chal$, **PA** runs the *Algorithm* 3 to generate an integrity proof $(\hat{\pi}, \hat{\mu})$, which will be issued on the blockchain. After that, he returns the challenged blocks $(\{h_{s-1}, ch_s, m_s, Y_s, r_s, \{C_i\}_{i \in \mathbf{P}_s}\}_{s \in B})$ to **VA** through a secure off-chain channel.
- **Verify**: Upon getting $(\hat{\pi}, \hat{\mu})$, if $C'_0 = C_0$, he checks the integrity of the blockchain ledger, as follows.

$$\prod_{\substack{i \in \mathbf{P}_s \\ s \in B}} e(C_i, g_2^{\sum_{c \in S_i} \alpha^{N+1-c} t_{i,c}})^{t_i} \stackrel{?}{=} e(\hat{\pi}, g_2) \cdot g_T^{\alpha^{N+1} \hat{\mu}}$$

If the above equation does not hold, **VA** aborts this process. Otherwise, by equation (12), he checks the correctness of the challenged blocks. If all challenged blocks are correct, **VA** outputs audit result $Result = 1$ to indicate that the blockchain ledger held by **PA** is intact and valid. Otherwise, outputs $Result = 0$. Finally, **VA** issues the audit result on the blockchain.

4 Optimized and Extended VRBCs

Benefits from the scalability of the underlying primitives, the proposed VRBC can also be optimized/extended to meet various service requirement.

4.1 VRBC With Optimized Auditing and Redaction

By observing BAT-based proof generation and redaction, we introduce two optimized strategies to improve the performance of blockchain auditing and redaction, respectively.

- *Optimized auditing*: Due to the structural characteristics of BAT, the costs (i.e., computation and communication) of blockchain auditing are determined by the node number of path union from challenged nodes to the root node. That is, the audit cost is minimized when all challenged blocks are on the same path or belong to the minimum path union. To this end, we define a new PRF $\phi_1^* : \{1, ..., q^l\} \times \mathbb{Z}_p^* \rightarrow [\frac{q(1-q^{l-1})}{1-q}, \frac{q(1-q^l)}{1-q}]$ where $l = \mathsf{level}(n,q)$, which enables **VA** to randomly pick $z^* \in [q^l]$ leaf nodes and challenge all path nodes from them to the root node. Besides, the coefficients of the challenged nodes are still calculated by $b_i = \phi_2(i, f_2)$. According to the audit challenge $Chal = (z^*, f_1, f_2)$ with ϕ_1^* and ϕ_2, **PA** generates an integrity proof by running *Algorithm* 3.
- *Delayed redaction*: Similarly, blockchain redaction requires updating the commitment of all path nodes from the redacted block to the root node, which means that redaction costs are also minimized when all redacted blocks are on the same path or adjacent paths. To this end, **SM** can adopt a delayed redaction strategy to achieve an efficient batch redaction. Specifically, he first divides BAT into a series of subregions based on leaf nodes. Then, he delays trivial redaction requests belonging to the same sub-region until an important request arises (e.g., a redaction request subject to law), and then redacts all the delayed blocks at once.

4.2 Transaction-Level VRBC

For achieving fine-grained redactions, we can extend the proposed scheme to a transaction-level version, as follows.

- *Chameleon hash*: To achieve transaction-level redaction, **BM** runs CH function to generate the chameleon hash value for each transaction Tx_i in B_s: he picks a random $r_i \in \mathbb{Z}_p^*$ and computes:

$$ch_i = g_1^{\mathsf{H}_1(Tx_i, Y) \cdot (x+y) + r_i} \tag{13}$$

  Then, he uses all chameleon hash values $\{ch_i\}$ to generate a MHT, whose root value m_s will be used to generate the block hash $h_s = \mathsf{H}_2(h_{s-1}, m_s, ctr_s)$.
- *Node commitment*: For verifiable redaction, one feasible solution is that **SM** adopts Pointproofs [37] to generate the transaction commitment TC_s for each block B_s. Further, he runs our commitment algorithm to calculate node commitment C_s with $(TC_s, C_{a_2}, ..., C_{a_{q+1}})$, instead of $(m_s, C_{a_2}, ..., C_{a_{q+1}})$.
- *Proof generation/verification*: The proof generation and verification of transaction-level redaction are similar to our basic scheme, except for the additional proof generation and verification of transaction commitment. Due to space limitations, we omit it here.
- *Transaction-level redaction*: Based on the above modifications, **SM** is able to calculate a collision (Tx'_i, Y'_s, r'_s) for the target transaction (Tx_i, Y_s, r_s). Then, he updates the transaction commitment of the redacted block and the path node commitments from the redacted node to the root node.

4.3 VRBC for Permissionless Setting

Similar to the scheme of Ateniese et al. [15], VRBC can also be implemented in a permissionless setting.

- *Chameleon hash*: By introducing a committee consisting of some important miners (e.g., the top 7 mining pools), VRBC allows them to execute a MPC protocol

to generate the hash key of the chameleon hash function, or redact on-chain data. For example, each party $P_i \in \{P\}_{i\in[1,n]}$ picks a random element $x_i, y_i \leftarrow \mathbb{Z}_p^*$. After that, they generate a uniform hash key pair (X, Y) via a MPC protocol.

$$X = \prod_{i=1}^{n} g_1^{x_i}, \quad Y = \prod_{i=1}^{n} g_1^{y_i} \tag{14}$$

Then, **BM** generate a new block $B_i = (ch_i, m_i, r_i)$ with (X, Y). To redact m_s with m'_s, each P_i generates a new ephemeral trapdoor piece $y'_{i,s} = \mathsf{H}_2(x_i, m'_s)$ and executes the MPC protocol to generate a new hash key $Y'_s = \prod_{i=1}^{n} g_1^{y_{i,s}}$. Thereafter, P_i computes the trapdoor piece $k_{i,s}$ and the randomness $r'_{i,s}$:

$$\begin{cases} k_{i,s} = \mathsf{H}_1(m_s, Y_s) \cdot (x_i + y_i) + \frac{r_s}{n} \bmod p \\ r'_{i,s} = k_{i,s} - \mathsf{H}_1(h_{s-1} || m'_s, Y'_s) \cdot (x_i + y'_{i,s}) \bmod p \end{cases}$$

Finally, they obtain the collision (ch_s, m'_s, Y'_s, r'_s) of block (ch_s, m_s, Y_s, r_s), where $r'_s = \sum_{i=1}^{n} r'_{i,s}$.

- *Vector commitment*: Due to the same main idea, our vector commitment algorithm can also be implemented in a decentralized committee according to Pointproofs [37]. Here, we omit the specific process.

5 Security Analysis

Theorem 1. ***(Correctness)****: The proposed scheme can achieve a correct verification in the block query and auditing phases.*

Proof. Due to the similar idea, the correctness of blockchain auditing can be deduced from the block query. Therefore, we will prove their correctness in the following two steps.

Step 1: we prove the correctness of block query process:

$$\begin{aligned} &\prod_{i\in\mathbf{P}_s} e(C_i, g_2^{\alpha^{N+1-c} t_i}) \\ &= \prod_{i\in\mathbf{P}_s} e(g_1^{\mathbf{v}_i^\top \mathbf{a} + \gamma_i}, g_2^{\alpha^{N+1-c} t_i}) \\ &= \prod_{i\in\mathbf{P}_s} e(g_1^{\mathbf{v}_i[-c]^\top \mathbf{a}[-c] + \gamma_i}, g_2^{\alpha^{N+1-c} t_i}) \cdot e(g_1^{v_{i,c}\alpha^c}, g_2^{\alpha^{N+1-c} t_i}) \\ &= e\left(\prod_{i\in\mathbf{P}_s} (C_i / g_1^{v_{i,c}\alpha^c})^{\alpha^{N+1-c} \cdot t_i}, g_2 \right) \cdot e(g_1^{\alpha^{N+1} \sum v_{i,c} t_i}, g_2) \\ &= e\left(\prod_{i\in\mathbf{P}_s} \pi_i, g_2 \right) \cdot g_T^{\alpha^{N+1} \cdot \sum v_{i,c} t_i} \end{aligned}$$

Distinctly, iff $\pi_s = \prod_{i\in\mathbf{P}_s} \pi_i$ and $\mu_s = \sum v_{i,c} t_i$, we get that

$$\prod_{i\in\mathbf{P}_s} e(C_i, g_2^{\alpha^{N+1-c} t_i}) = e(\pi_s, g_2) \cdot g_T^{\alpha^{N+1} \mu_s} \tag{15}$$

Step 2: Then we analyze the correctness of the blockchain auditing. Let $t_{i,c}$ be the coefficient of the c -th element of the vector $\mathbf{v}_i$. From equation (15), we can get that:

$$\begin{aligned} &\prod_{\substack{i\in\mathbf{P}_s \\ s\in B}} e(C_i, g_2^{\sum_{c\in S_i} \alpha^{N+1-c} t_{i,c}})^{t_i} \\ &= \prod_{\substack{i\in\mathbf{P}_s \\ s\in B}} \prod_{c\in S_i} e(C_i, g_2^{\alpha^{N+1-c} t_{i,c} t_i}) \\ &= e(\prod_{\substack{i\in\mathbf{P}_s \\ s\in B}} \pi_i, g_2) \cdot g_T^{\alpha^{N+1} \sum v_{i,c} t_{i,c} t_i} \end{aligned} \tag{16}$$

Distinctly, iff $\hat{\pi} = \prod \pi_i$ and $\hat{\mu} = \sum v_{i,c} t_{i,c} t_i$, we capture that:

$$\prod_{\substack{i\in\mathbf{P}_s \\ s\in B}} e(C_i, g_2^{\sum_{c\in S_i} \alpha^{N+1-c} t_{i,c}})^{t_i} = e(\hat{\pi}, g_2) \cdot g_T^{\alpha^{N+1} \hat{\mu}} \tag{17}$$

In summary, the proposed scheme can guarantee the correctness of block query and blockchain auditing. □

Theorem 2. ***(Soundness)****: As long as the ℓ-wBDHE problem is hard in bilinear group, in the algebraic group model, no PPT adversary can pass the block query and blockchain auditing protocol of the proposed scheme with a non-negligible probability.*

Proof. Essentially, block query is a special case of blockchain auditing, so we only discuss the soundness of the latter. For clarity, we set a series of soundness games, as follows.

Game 0. The initial game **Game 0** is an interactive soundness game, as defined in Section 2.3.

Game 1. This game is essentially identical to **Game 0**, except that the challenger $\mathcal{C}$ maintains a list of all issued commitments. Once the adversary $\mathcal{A}$ queries a commitment C_i that is not part of the list, $\mathcal{C}$ aborts it and declares failure.

Game 2. This game is essentially identical to **Game 1**, except that $\mathcal{C}$ maintains a list of its responses to the Open query made by $\mathcal{A}$. For a challenge $Chal$, if $\mathcal{A}$ can win this game by returning two different proofs $(\hat{\pi}, \hat{\pi}')$ with a non-negligible probability ϵ, $\mathcal{C}$ aborts it and declares failure.

To analyze the difference in success probability between **Game 1** and **Game 2**, we assume that the challenged BAT consists of n blocks, where each block B_i contains data m_i along with the commitment C_i issued by Commit. Furthermore, let $Chal = \{(i, b_i)\}$ be the challenge that causes the failure. Then, we suppose that the expected response generated by an honest prover is $\hat{\pi} = \prod \pi_i$ and $\hat{\mu} = \sum v_{i,c}, t_{i,c}, t_i$, where $c \in S_i, i \in \mathbf{P}_s, s \in B \in Chal$, while the response generated by $\mathcal{A}$ are $\hat{\pi}'$ and $\hat{\mu}'$. Based on the correctness demonstrated in *Theorem 1*, the expected response will satisfy the following verification equation:

$$\prod_{\substack{i\in\mathbf{P}_s, \\ s\in B}} e(C_i, g_2^{\sum_{c\in S_i} \alpha^{N+1-c} t_{i,c}})^{t_i} = e(\hat{\pi}, g_2) \cdot g_T^{\alpha^{N+1} \hat{\mu}} \tag{18}$$

Likewise, the response $\hat{\pi}' (\neq \hat{\pi})$ that causes the failure also satisfies the verification equation, i.e., that

$$\prod_{\substack{i\in\mathbf{P}_s, \\ s\in B}} e(C_i, g_2^{\sum_{c\in S_i} \alpha^{N+1-c} t_{i,c}})^{t_i} = e(\hat{\pi}', g_2) \cdot g_T^{\alpha^{N+1} \hat{\mu}'} \tag{19}$$

Distinctly, $\hat{\mu}' \neq \hat{\mu}$, otherwise it means that $\hat{\pi}' = \hat{\pi}$, which contradicts our assumption above. Further, we prove that we can build a simulator $\mathcal{S}$ to solve the ℓ-wBDHE problem: $\mathcal{S}$ behaves like $\mathcal{C}$ in **Game 1**, except for giving black-box access to $\mathcal{A}$. From equation (18) and (19), $\mathcal{S}$ can get that:

$$\begin{aligned} e(\hat{\pi}/\hat{\pi}', g_2) &= g_T^{\alpha^{N+1}(\hat{\mu}'-\hat{\mu})} \\ &= e(g_1^{\alpha^{N+1}(\hat{\mu}'-\hat{\mu})}, g_2) \end{aligned} \tag{20}$$

Now, $\mathcal{S}$ get that $(\hat{\pi}/\hat{\pi}')^{(\hat{\mu}'-\hat{\mu})^{-1}} = g_1^{\alpha^{N+1}}$. Thus, if there is a non-negligible difference between the success probability of $\mathcal{A}$ in **Game 1** and **Game 2**, we can build a simulator $\mathcal{S}$ to utilize $\mathcal{A}$ to solve ℓ-wBDHE problem.

Game 3. This game is essentially identical to **Game 2**, except that $\mathcal{C}$ can track Open queries and interactive instances. In this case, if there are some responses $\hat{\mu}$ that can pass the verification, but at least one of the aggregated proofs $\hat{\mu}'$ is not equal to the expected proof $\hat{\mu} = \sum v_{i,c}, t_{i,c}, t_i$, $\mathcal{C}$ aborts it and declares failure.

Based on the same assumption as **Game 2**, we can also build a simulator $\mathcal{S}$ to solve the ℓ-wBDHE problem. In fact, $\mathcal{A}$ can win **Game 3** in one of the following three cases:

- **Case 1**: "H-lucky" queries: For any queries $(, C, S, \mathbf{v}[S])$ launched by the algebraic $\mathcal{A}$ [38], who must output $\mathbf{b} \in \mathbb{Z}_p^N$ and $\mathbf{d} \in \mathbb{Z}_p^{N-1}$, such that

$$\begin{aligned} C &= g_1^{\mathbf{b}^\top \mathbf{a} + \alpha^N \mathbf{d}^\top \mathbf{a}[-1]} \\ &= g_1^{\sum_{i\in[N]} b_i \alpha^i + \sum_{j\in[N-1]} d_j \alpha^{N+1+j}} \end{aligned} \tag{21}$$

  On this basis, a query is "H-lucky" if and only if:

$$\mathbf{v}[S] \not\equiv_p \mathbf{b}[S] \quad \text{and} \quad (\mathbf{v}[S] - \mathbf{b}[S])^\top \mathbf{t} \equiv_p 0 \tag{22}$$

  where $\mathbf{t} = (\mathsf{H}_2(i, C, \mathbf{v}[i]))_{i\in S}$. Notably, a query is "H-lucky" with a negligible probability, at most $1/p$.
- **Case 2**: "H′-lucky" challenge: For any challenges $(*, \{C_i, S_i, \mathbf{v}_i[S_i]\}_{i\in\mathbf{P}_s, s\in B})$ launched by $\mathcal{A}$, it must output some $\{\mathbf{b}_i, \mathbf{d}_i\}_{i\in\mathbf{P}_s, s\in B}$, such that.

$$C_i = g_1^{\mathbf{b}_i^\top \mathbf{a} + \alpha^N \mathbf{d}_i^\top \mathbf{a}[-1]} \tag{23}$$

  Similarly, a challenge is "H′-lucky" if $\exists i$,

$$\begin{aligned} &(\mathbf{v}_i[S_i] - \mathbf{b}_i[S_i])^\top \mathbf{t}_i[S_i] \not\equiv_p 0 \\ &\sum_{i\in\mathbf{P}_s, s\in B} (\mathbf{v}_i[S_i] - \mathbf{b}_i[S_i])^\top \mathbf{t}_i[S_i] t_i \equiv_p 0 \end{aligned} \tag{24}$$

  where $\mathbf{t}_i[S_i] = (t_{i,c})_{c\in S_i} = (\mathsf{H}_2(c, C_i, \mathbf{v}_i[S_i]))_{c\in S_i}$. Obviously, the challenge is "H′-lucky" with probability at most $1/p$, which is negligible.
- **Case 3**: "Solving ℓ-wBDHE problem": Consider the output $(\hat{\pi}, \{C_i, \{S_i^z, \mathbf{v}_i^z[S_i^z]\}_{z=0,1}\}_{i\in\mathbf{P}_s})$ of a winning adversary $\mathcal{A}$, who must output $\mathbf{b}_i, \mathbf{d}_i$, such that:

$$C_i = g_1^{\mathbf{b}_i^\top \mathbf{a} + \alpha^N \mathbf{d}_i^\top \mathbf{a}[-1]} \tag{25}$$

  Since $\mathbf{v}_i^0[S_i^0 \cap S_i^1] \neq \mathbf{v}_i^1[S_i^0 \cap S_i^1]$,it must be the case that either $\mathbf{v}_i^0[S_i^0] \neq \mathbf{b}_i[S_i^0]$ or $\mathbf{v}_i^1[S_i^1] \neq \mathbf{b}_i[S_i^1]$. Therefore, let $(S_i^*, \mathbf{v}_i^*, \hat{\pi}^*)$ be an accepting instance, where $*$ is equal to 0 or 1, and $\mathbf{v}_i^*[S_i^*] \neq \mathbf{b}_i[S_i^*]$, we have:

$$\prod_{\substack{i\in\mathbf{P}_s \\ s\in B}} e(C_i, g_2^{\sum_{c\in S_i^*} \alpha^{N+1-c} t_{i,c}})^{t_i} = e(\hat{\pi}^*, g_2) \cdot g_T^{\alpha^{N+1}\hat{\mu}^*} \tag{26}$$

  where $\hat{\mu}^* = \mathbf{v}_i^*[S_i^*]^\top \mathbf{t}_i[S_i^*] t_i$. This implies that

$$\begin{aligned} &g_1^{\sum_{i\in\mathbf{P}_s}^{s\in S_i} \sum_{c\in S_i} \alpha^{N+1-c} \mathbf{b}_i^*[-c]^\top \mathbf{a}[-c] t_i, t_{i,c}} \cdot \\ &\left(g_1^{\sum_{i\in\mathbf{P}_s}^{s\in S_i} \alpha^N \mathbf{d}_i^\top \mathbf{a}[-1] \sum_{c\in S_i^*} \alpha^{N+1-c} t_i, t_{i,c}} \right) \cdot (\hat{\pi}^*)^{-1} \\ &= g_1^{\alpha^{N+1} \sum_{i\in\mathbf{P}_s}^{s\in S_i} (\mathbf{v}_i^*[S_i^*] - \mathbf{b}_i^*[S_i^*])^\top \mathbf{t}_i[S_i^*] t_i} \end{aligned} \tag{27}$$

  Since there are no "H-lucky" queries, we have:

$$(\mathbf{v}_i^*[S_i^*] - \mathbf{b}_i^*[S_i^*])^\top \mathbf{t}_i[S_i^*] \not\equiv_p 0 \tag{28}$$

  Also, there are no "H′-lucky" challenges, we have

$$\Delta = \sum_{i\in\mathbf{P}_s, s\in B} (\mathbf{v}_i^*[S_i^*] - \mathbf{b}_i^*[S_i^*])^\top \mathbf{t}_i[S_i^*] t_i \not\equiv_p 0 \tag{29}$$

  Meanwhile, the LHS of Equation (27) can be calculated with $(g_1^{\mathbf{a}}, g_1^{\alpha^N \mathbf{a}[-1]}, g_1^{\alpha^{2N}\mathbf{a}})$. Therefore, $\mathcal{S}$ can calculate $\psi \not\equiv_p \Delta^{-1}$ and raise both sides of Equation (27) to the power ψ to compute $g_1^{\alpha^{N+1}}$.

Therefore, if there is a non-negligible difference between the success of $\mathcal{A}$ in **Game 2** and **Game 3**, we can build a simulator $\mathcal{S}$ to employ $\mathcal{A}$ to solve ℓ-wBDHE problem. □

Theorem 3. ***(Controlled redaction)****: The proposed scheme only allows the redactor holding a trapdoor key to redact the blockchain and prevents adversaries from exploiting collisions to extract the trapdoor key or forge a new collision.*

Proof. By adopting Chen et al.'s [36] double-trapdoor chameleon hash family, the proposed scheme enables the redactor to achieve a controlled redaction on the blockchain. That is, the proposed scheme inherits the security of [36], involving collision resistance, trapdoor collisions, and key-exposure freeness. The former two ensure that only the redactor can compute a collision to redact blockchain. In addition, the last one prevents adversaries from extracting the trapdoor key with collisions. Due to space constraints, the specific proof is omitted here. □

6 Performance Evaluation

In this section, we evaluate the performance of the proposed scheme from both theoretical analysis and implementation.

6.1 Theoretical Analysis

For better performance evaluation, we select Ethereum [2] and AMVA17 [15] as compared schemes. Meanwhile, we define some notations in Table 2, which will be used to

TABLE 2
Notations

Notation	Description
n	Number of blocks in the blockchain
n_q	Number of blocks before the queried block
m	Number of transactions in a standard block
q	Number of forks in the q-ary BAT
N	Number of dimensions of the family vector
ρ	Number of nodes in the audited path union
l	Height of the MHT in the block, i.e., $\mathsf{level}(m, 2)$
ℓ	Height of the redacted/queried node in BAT.
$\mathcal{H}$	Cost of running a hash function H_1
H	Cost of running a standard hash function H_2
M	Cost of mining a standard block
Exp	Cost of running a exponentiation operation
Pair	Cost of running a bilinear pairing operation
$\lvert\mathbb{G}\rvert$	Size of the element in $\mathbb{G}$
$\lvert\mathbb{Z}_p\rvert$	Size of the element in $\mathbb{Z}_p$
$\lvert$Tx$\rvert$	Size of the transaction in a standard block
$\lvert$BH$\rvert$	Size of the block header in a standard block

evaluate the computation, communication, and storage costs of compared schemes in Tables 3 and 4, respectively.

During the system setup phase, all compared schemes require 1M to generate a standard genesis block. In addition, AMVA17 consumes $1\mathsf{H} + 3\mathsf{Exp}$ to generate the parameters of the chameleon hash function, while our scheme requires additional cost $2q\mathsf{H} + 3N\mathsf{Exp}$ to generate the parameters of chameleon hash and vector commitment. Correspondingly, AMVA17 and our scheme require extra $2(|\mathbb{Z}_p| + |\mathbb{G}|)$ and $4(N + 1)|\mathbb{G}|$ bandwidth to publish public parameters on the blockchain, respectively. Although the additional computation and communication costs of our scheme are higher than that of AMVA17, it is negligible relative to the cost of generating a standard block, which is necessary and acceptable for our scheme to achieve efficient validation.

In the block append phase, except for generating a standard block with 1M, the block proposer **BM** in AMVA17 consumes extra $1\mathsf{H} + 2\mathsf{Exp}$ to calculate the chameleon hash value for each block. Compared to AMVA17, our scheme requires extra $q(2\mathsf{H} + \mathsf{Exp})$ to bind a new block to the q-ary BAT, which enables other **BM**s to check the correctness and validity of on-chain block data. Accordingly, our scheme requires additional computation costs $(q-1)\mathsf{H} + 1\mathcal{H} + (N-1)\mathsf{Exp}$ to verify the correctness and validity of the new block, as well as additional bandwidth $q|\mathbb{G}|$ to publish its family node commitments on the blockchain.

Concerning block redaction, only AMVA17 and our scheme allow the redactor to redact on-chain data. The redaction cost of the former is $1\mathsf{H} + 1\mathsf{Exp}$, while that of the latter is $2\mathcal{H} + (\ell + 1)(\mathsf{H} + \mathsf{Exp})$, in which $\ell(\mathsf{H} + \mathsf{Exp})$ is used to update the node commitment from the redacted block to the root node in BAT. Correspondingly, other **BM**s in our scheme also require $\ell(\mathsf{H} + \mathsf{Exp})$ to update the related commitments and calculate the updated root node commitment to check the validity of the redacted block. Let us note that the communication cost of our scheme is almost equal to AMVA17 since our scheme allows **BM**s to update the redacted block and BAT by themselves.

Regarding block query, any continuously-online LN can easily verify the validity of the queried block with $2^{l+1}\mathsf{H}$ in Ethereum, $(2^{l+1} + 1)\mathsf{H} + 2\mathsf{Exp}$ in AMVA17, and $2^{l+1}\mathsf{H} + 1\mathcal{H}$ in our scheme, respectively. Without loss of generality, Tables 3 and 4 summarize the query costs of rejoined LN (like a new node). Specifically, Ethereum (AMVA17) requires **VA** to download all blocks from the queried block to the genesis block (the entire blockchain ledger), and verify them to capture the correctness and validity of the queried block. Unlike them, our scheme binds the blockchain as a q-ary BAT, which enables **PA** to generate a validity proof for the queried block, and **VA**s only need to download and validate the queried block along with its proof. Notably, similar to the proof generation in PointProofs[37], the **PA** in our scheme can first calculate the sum of coefficients of each parameter and then perform exponential operations, which makes the cost of proof generation at most $q\ell\mathsf{H} + (2N - 1)\mathsf{Exp}$. Therefore, both the computation and communication costs of our scheme are much lower than those of Ethereum and AMVA17.

Taking into account the auditing of the blockchain, similar to the block query, **VA** in Ethereum and AMVA17 needs to download and verify the entire blockchain ledger held by **PA**. Nevertheless, our scheme adopts an optimized auditing strategy, which enables **VA** to check the integrity of the blockchain only by verifying the blocks of path union determined by the audit challenge. Moreover, both the FNs and the LNs of our scheme require only an additional $|\mathbb{Z}_p| + 3|\mathbb{G}|$ to store metadata for each block, which is much less than the size of a standard block.

In practice, block query and blockchain auditing are very frequent, which means that the additional costs of our scheme at each phase are negligible and acceptable relative to the benefits obtained by achieving efficient verification.

6.2 Implementation

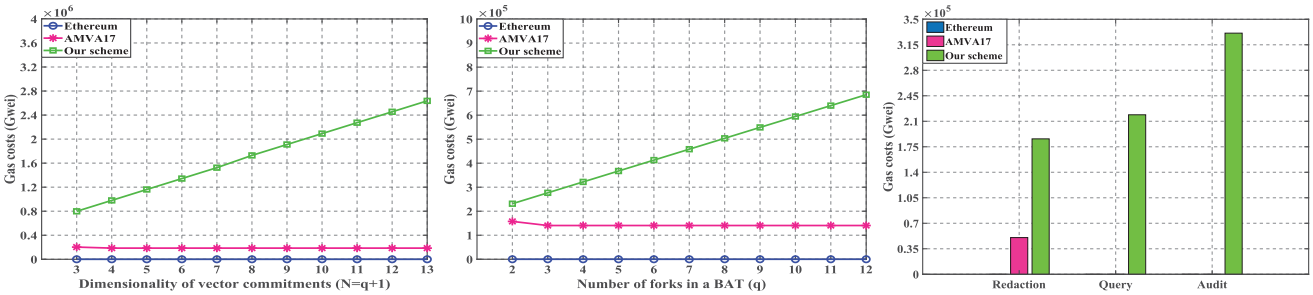
To obtain a visual performance evaluation, we set up a series of experiments to estimate the on-chain and off-chain costs of compared schemes at different phases.

TABLE 3
Comparisons of Computation Costs

Schemes	System setup	Block append		Block redaction		Block query		Blockchain auditing	
		Bind	verify	Redact	verify	Prove	Verify	Prove	Verify
Ethereum [2]	1M	1M	2^{l+1}H	–	–	–	$n_q(2^{l+1}\mathsf{H})$	–	$n(2^{l+1}\mathsf{H})$
AMVA17 [15]	1M+ 1H +3Exp	1M+1H +2Exp	$(2^{l+1}+1)$H+2Exp	1H+1Exp	$(2^{l+1}+1)$H +2Exp	–	$n(2^{l+1}+1)$H+2nExp	–	$n(2^{l+1}+1)$H+2nExp
Our scheme	1M+2qH +3NExp	1M+1$\mathcal{H}$+q(2H +Exp)	$(2^{l+1}+q)$H+1$\mathcal{H}$ +$(N+1)$Exp	2$\mathcal{H}$+$(\ell+1)$ (H+Exp)	$(2^{l+1}+\ell)$H+1$\mathcal{H}$ +$(\ell+1)$Exp	$q\ell$H+$(2N$- 1)Exp	2^{l+1}H+1$\mathcal{H}$+$(\ell+2)$ (Exp+Pair)	$q\rho$H+$(2N$- 1)Exp	$\rho(\mathcal{H} + 2^{l+1}\mathsf{H})$ $(\rho+1)$(Exp+Pair)

TABLE 4
Comparisons of Communication and Storage Costs

Schemes	Communication Costs					Storage costs	
	Setup	Append	Redaction	Query	Auditing	FN	LN
Ethereum [2]	$1\|\mathsf{BH}\|+m\|\mathsf{Tx}\|$	$1\|\mathsf{BH}\|+m\|\mathsf{Tx}\|$	–	$n_q(\|\mathsf{BH}\|+m\|\mathsf{Tx}\|)$	$n(\|\mathsf{BH}\|+m\|\mathsf{Tx}\|)$	$n(\|\mathsf{BH}\|+m\|\mathsf{Tx}\|)$	$n\|\mathsf{BH}\|$
AMVA17 [15]	$1\|\mathsf{BH}\|+m\|\mathsf{Tx}\|+2$ $(\|\mathbb{Z}_p\|+\|\mathbb{G}\|)$	$1\|\mathsf{BH}\|+m\|\mathsf{Tx}\|+2\|\mathbb{Z}_p\|$ $+1\|\mathbb{G}\|$	$1\|\mathsf{BH}\|+m\|\mathsf{Tx}\|$ $+3\|\mathbb{Z}_p\|$	$n(\|\mathsf{BH}\|+m\|\mathsf{Tx}\|)$ $+n(2\|\mathbb{Z}_p\|+\|\mathbb{G}\|)$	$n(\|\mathsf{BH}\|+m\|\mathsf{Tx}\|)$ $+n(2\|\mathbb{Z}_p\|+\|\mathbb{G}\|)$	$n(\|\mathsf{BH}\|+m\|\mathsf{Tx}\|)$ $+n(2\|\mathbb{Z}_p\|+\|\mathbb{G}\|)$	$n(\|\mathsf{BH}\|)$ $+n(2\|\mathbb{Z}_p\|+\|\mathbb{G}\|)$
Our scheme	$1\|\mathsf{BH}\|+m\|\mathsf{Tx}\|+4$ $(N+1)\|\mathbb{G}\|$	$1\|\mathsf{BH}\|+m\|\mathsf{Tx}\|+2\|\mathbb{Z}_p\|$ $+(q+1)\|\mathbb{G}\|$	$1\|\mathsf{BH}\|+m\|\mathsf{Tx}\|+2$ $(\|\mathbb{Z}_p\|+\|\mathbb{G}\|)$	$1\|\mathsf{BH}\|+m\|\mathsf{Tx}\|+4\|\mathbb{Z}_p\|$ $+(\ell+4)\|\mathbb{G}\|$	$\rho(\|\mathsf{BH}\|+m\|\mathsf{Tx}\|)+(3\rho+1)$ $(\|\mathbb{Z}_p\|+\|\mathbb{G}\|)$	$n(\|\mathsf{BH}\|+m\|\mathsf{Tx}\|)$ $+n(\|\mathbb{Z}_p\|+3\|\mathbb{G}\|)$	$n\|\mathsf{BH}\|)+n(\|\mathbb{Z}_p\|$ $+3\|\mathbb{G}\|)$



(a) System setup (b) Block append (c) Other operations

Fig. 4. Comparisons of on-chain costs.

We adopt Python to implement the above compared schemes by calling the GNU Multiple Precision Arithmetic (GMP) Library[3], Pairing Based Cryptography (PBC) Library[4] and Fast Elliptic Curve Cryptography (fastecdsa 2.2.3) Library[5]. For each scheme, we test the on-chain costs (i.e., gas consumption[6]) of various operations on the official public test-net Ropsten[7] of Ethereum. Meanwhile, we also test the off-chain computation costs of all compared schemes on CentOS 8.3.2011 with two 2.20GHz Intel Xeon Silver 4210 CPU and 128GB memory. Specifically, we adopt SHA-256 as the standard hash function to generate block/transaction hash value. Besides, each test block instance contains 170 transactions of size 660 Bytes[8] and all experiments will be repeated 30 times to obtain average results.

6.2.1 On-Chain Costs

As shown in Fig. 4, we test the additional on-chain costs (i.e., gas costs) of compared schemes incurred by blockchain redaction and efficient verification.

During the system setup phase, additional gas will be used to issue some necessary parameters. Specifically, the extra gas cost of AMVA17 is constant 0.203×10^6 Gwei, while the extra gas cost of our scheme increases linearly with vector dimension: 0.798×10^6 Gwei, 1.343×10^6 Gwei, and 2.273×10^6 Gwei for $N = 3, 6, 11$, respectively. Similarly, in the block append phase, AMVA17 consumes a constant 0.158×10^6 Gwei gas to publish some metadata for correctness verification. Meanwhile, our scheme consumes linearly increasing gas costs to publish necessary metadata for correctness and validity verification: 0.231×10^6 Gwei, 0.367×10^6 Gwei, and 0.594×10^6 Gwei for $q = 2, 5, 10$, respectively. To achieve elegant redaction, AMVA17 consumes only 0.050×10^6 Gwei to issue an updated (collision) block, while our scheme requires 0.186×10^6 Gwei, most of which is used to publish the latest vector commitments of the redacted block and the BAT root node (i.e., C'_s and C'_0).

Considering efficient validation in redactable blockchain, only our scheme can achieve efficient block query and blockchain auditing. As shown in Fig. 4c, each block query process in our scheme requires a constant gas 0.218×10^6 Gwei, of which 46.3% gas is borne by **VA** for publishing the query request and verification result. For each blockchain audit process, our scheme consumes additional gas 0.331×10^6 Gwei, where 64.4% gas is borne by **VA** for issuing the audit challenge and the result on the blockchain.

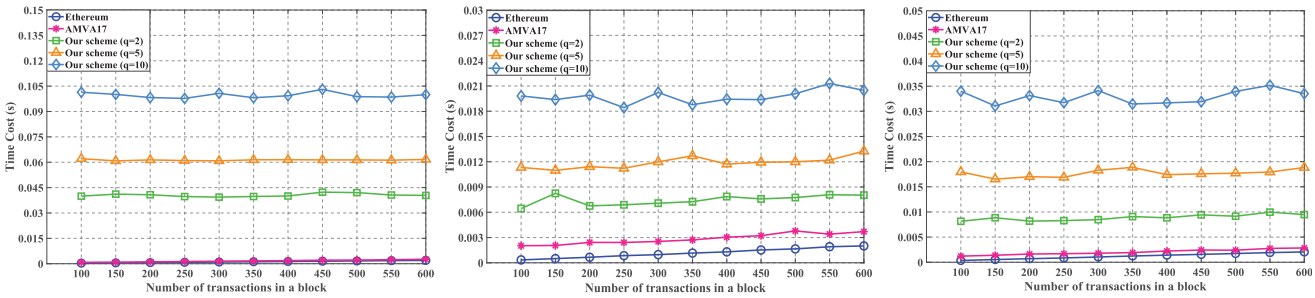
Although our scheme requires more extra gas than AMVA17 at all phases, it is necessary to provide efficient validity verification in block query and blockchain auditing scenarios. Moreover, our query and auditing protocol share all "challenge-response" instances on the blockchain, which avoids the on/off-chain resource waste caused by repeated audit requests within the same period.

6.2.2 Off-Chain Costs

For better evaluation, we also estimate the off-chain computation costs (i.e., time costs) of compared schemes.

In the system setup phase, the time cost is used to generate the parameters pp and the genesis block. From Fig. 5a,

3. https://gmplib.org/.
4. https://crypto.stanford.edu/pbc/download.html.
5. https://pypi.org/project/fastecdsa/
6. Gas is the basic unit used to pay for various computation operations when conducting transactions or executing smart contracts in the Ethereum ecosystem, (1×10^9Gwei $= 1$Ether $= \$2992$, April 22, 2022).
7. https://ropsten.etherscan.io/
8. This is the average of the parameters for the latest 10,000 blocks in Ethereum as of April 22, 2022.



(a) System setup (b) Block append (generation) (c) Block append (verify)

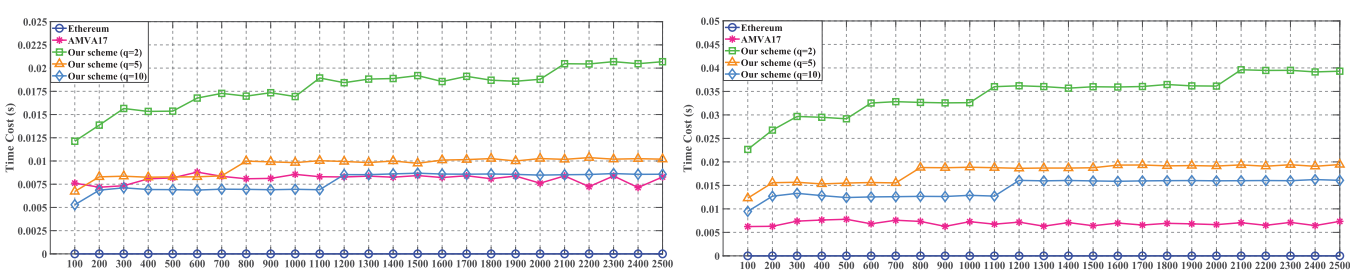
Fig. 5. Comparisons of off-chain computation costs in the system setup and block append phases.

Ethereum requires 0.0003s, 0.001s and 0.0016s to package 100, 300 and 500 transactions, respectively. The corresponding costs of AMVA17 are 0.0008s, 0.0015s and 0.0022s, of which the extra time 0.0005s is used to generate the parameters of the chameleon hash function. Similarly, our scheme separately requires extra time 0.0405s, 0.0613s, and 0.0996s during $q = 2, 5, 10$ to initialize the chameleon hash function and vector commitment algorithm. Distinctly, the system setup cost of all compared schemes mainly depends on the number of parameters rather than transactions.

During the block append phase, the time costs consist of block generation and verification. In Fig. 5b, Ethereum requires 0.0003s, 0.001s, and 0.0017s to package 100, 300 and 500 transactions into a new block, respectively. On this basis, AMVA17 requires additional 0.0017 seconds to generate the chameleon hash value, while our scheme requires extra 0.006s, 0.011s, and 0.019s during $q = 2, 5, 10$ to generate the chameleon hash value and commitments. From Fig. 5c, Ethereum consumes 0.0004s, 0.001s and 0.0017s to verify blocks containing 100, 300 and 500 transactions, respectively. On this basis, AMVA17 requires extra 0.001s to verify the correctness of the new block, while our scheme requires extra 0.008s, 0.017s, and 0.031s during $q = 2, 5, 10$ to check the correctness and validity of the new block.

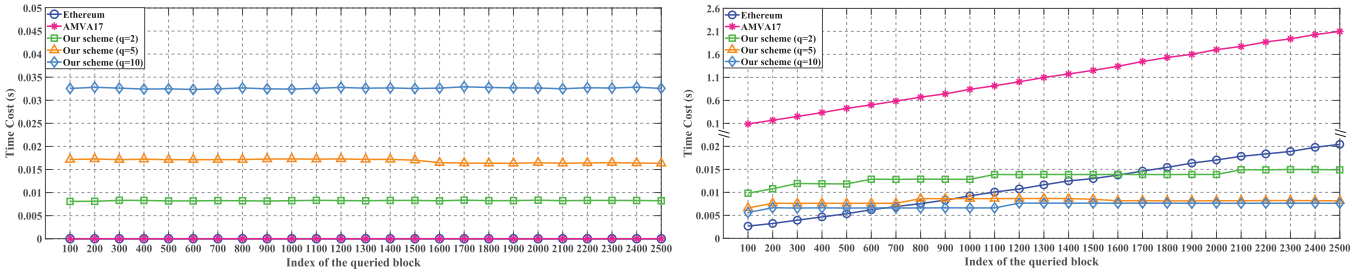
Regarding block redaction, in Fig. 6, AMVA17 requires a constant time 0.008s to redact the target block via generating a collision block. Correspondingly, the verification cost is also 0.008s. However, both the redaction and verification costs of our scheme show a stair-like growth trend as the block index increases (i.e., efficient redaction and verification with sublinear cost), which is determined by the number of BAT forks and the index of the redacted block. As regards redaction costs, our scheme redacts the 500th block with 0.015s, 0.008s and 0.006s during $q = 2, 5, 10$, respectively. Besides, the redaction costs of the 2000th block are 0.019s, 0.010s, and 0.008s. With regard to verification costs, our scheme separately requires 0.029s 0.015s and 0.012s to check the 500th block during $q = 2, 5, 10$, and the corresponding verification cost of the 2000th block are 0.036s, 0.019s and 0.016s. Notably, both the redaction and verification costs of our scheme decrease as q increases, which is opposite to the cost trend shown in Fig. 5. Meanwhile, as q increases, the cost of our scheme approaches AMVA17, which means that our scheme can achieve additional efficient verification with the same costs as AMVA17.

Taking into account the block query costs shown in Fig. 7, both Ethereum and AMVA17 only need to directly return the relevant blockchain data, so their proof generation cost shown in Fig. 7a is 0s. Unlike them, our scheme requires **PA** to generate a proof whose time costs depend only on the parameter q : 0.008s, 0.017s and 0.032s during $q = 2, 5, 10$. Recalling Table 4, both the communication costs of Ethereum and AMVA17 are much higher than our scheme, so the additional cost of proof generation in our scheme is negligible and acceptable. Furthermore, Fig. 7b shows that the verification costs of all compared schemes increase linearly with the index of the queried block. Specifically, AMVA17 separately requires 0.427s and 1.704s to check the validity of blocks 500 and 2000, which are much higher than that of Ethereum and our scheme. Furthermore, to check the 500-th block, Ethereum requires 0.005s, our scheme needs 0.011, 0.008s and 0.006s during $q = 2, 5, 10$, respectively. In contrast, the verification costs for the 2000th block in Ethereum is 0.017s, which is higher than our scheme (0.014s, 0.008s, 0.007s for $q = 2, 5, 10$). As a result,



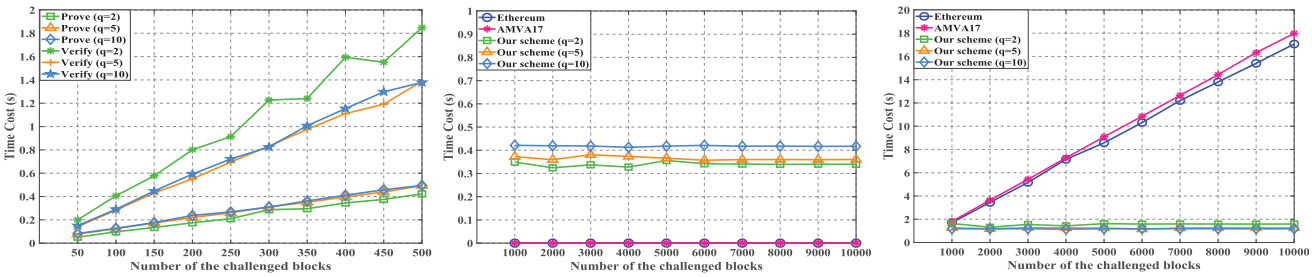
(a) Redaction (b) Verification

Fig. 6. Comparisons of off-chain computation costs in the block redaction phases.



(a) Proof generation (b) Proof verification

Fig. 7. Comparisons of off-chain computation costs in the block query phase.



(a) Our auditing (b) Proof generation (c) Proof verification

Fig. 8. Comparisons of off-chain computation costs in the blockchain auditing phase.

our scheme achieves efficient block query with sublinear costs in the redactable blockchain.

Regarding blockchain auditing, recalling Table 1, only our scheme can achieve integrity auditing for the blockchain ledger, while Ethereum and AMVA17 have to download and verify the entire ledger to capture its integrity. Thus, we first test the time costs of proof generation and verification in our scheme. Based on Ateniese et al.'s work [24], if the corrupted block rate of the checked file is 1%, the precision rate of integrity auditing is 95% during challenging 300 blocks and 99% during challenging 460 blocks. As depicted in Fig. 8a, the proof generation costs of challenging 300 blocks in our scheme are 0.286s, 0.310s and 0.309s during $q = 2, 5, 10$, while the corresponding costs of challenging 460 blocks are 0.375s, 0.438s, and 0.457s, respectively. In addition, for $q = 2, 5, 10$, the verification costs of our scheme are 1.227s, 0.831s, and 0.825s during challenging 300 blocks, as well as 1.551s, 1.192s, and 1.298s during challenging 460 blocks.

Furthermore, we set the number of challenged blocks at 460 in our scheme and further evaluate its performance by comparing it with other schemes. As shown in Fig. 8b, **PA** in Ethereum and AMVA17 only returns the entire ledger directly (i.e., the proof generation cost is 0s). To achieve efficient integrity auditing, our scheme consumes 0.339s, 0.365s, and 0.418s to generate integrity proof during $q = 2, 5, 10$, respectively. Correspondingly, the verification costs of our scheme are 1.549s, 1.239s, and 1.178s. Nevertheless, the verification costs for Ethereum and AMVA17 increase linearly with the ledger scale of the blockchain. Specifically, to check a blockchain ledger containing 5000 blocks, Ethereum and AMVA17 require 8.596s and 9.090s, respectively. When the audited ledger consists of 10000 blocks, Ethereum and AMVA17 require 17.070s and 17.981s, respectively. Distinctly, the auditing costs of Ethereum and AMVA17 are much higher than our scheme, which means that the method of obtaining blockchain integrity by verifying all blocks of the ledger is unfeasible, especially for the resource-constrained **VA** in some frequent audit scenarios.

7 Conclusion

Focusing on the security issues of redactable blockchains, we design a novel authentication data structure BAT to provide efficient validity verification for redactable blockchains. On this basis, we propose an efficient block query protocol, in which resource-constrained LNs can query and validate the blockchain data held by FNs at an acceptable cost. Furthermore, the first auditing protocol for redactable blockchain is designed to verify the integrity of the blockchain ledger held by FNs. Let us note that our auditing protocol can also be used as a subroutine of the consensus mechanism to prevent malicious miners from appending blocks to the blockchain. Furthermore, we design some optimized strategies to reduce the computation costs of auditing and redaction of the proposed VRBC scheme, and extend it to the transaction level and permissionless versions. In conclusion, the proposed scheme is the first verifiable redactable blockchain supporting efficient data query and auditing, which is significant for improving the security and practicability of redactable blockchains.

References

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: http://bitcoin.org/bitcoin.pdf
[2] G. Wood et al., "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, vol. 151, no. 2014, pp. 1–32, 2014.

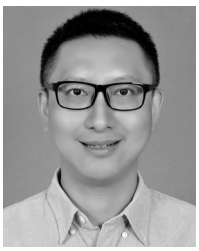
[3] M. Möser et al., "An empirical analysis of traceability in the monero blockchain," 2017, *arXiv:1704.04299*.
[4] M. Raikwar, S. Mazumdar, S. Ruj, S. S. Gupta, A. Chattopadhyay, and K.-Y. Lam, "A blockchain framework for insurance processes," in *Proc. 9th IFIP Int. Conf. New Technol., Mobility Secur.*, 2018, pp. 1–4.
[5] K. Korpela, J. Hallikas, and T. Dahlberg, "Digital supply chain transformation toward blockchain integration," in *Proc. 50th Hawaii Int. Conf. Syst. Sci.*, 2017, pp. 4182–4191.
[6] S. Krenn, H. C. Pöhls, K. Samelin, and D. Slamanig, "Chameleon-hashes with dual long-term trapdoors and their applications," in *Proc. Int. Conf. Cryptol. Afr.*, 2018, pp. 11–32.
[7] H. Yuan, X. Chen, J. Wang, J. Yuan, H. Yan, and W. Susilo, "Blockchain-based public auditing and secure deduplication with fair arbitration," *Inf. Sci.*, vol. 541, pp. 409–425, 2020.
[8] G. Tian et al., "Blockchain-based secure deduplication and shared auditing in decentralized storage," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 6, pp. 3941–3954, Nov./Dec. 2021.
[9] X. Ma, C. Wang, and X. Chen, "Trusted data sharing with flexible access control based on blockchain," *Comput. Standards Interfaces*, vol. 78, 2021, Art. no. 103543.
[10] C. Hopkins, "If you own bitcoin, you also own links to child porn," 2015. [Online]. Available: https://www.dailydot.com/business/bitcoin-child-porn-transaction-code/
[11] R. Matzutt et al., "A quantitative analysis of the impact of arbitrary blockchain content on bitcoin," in *Proc. Int. Conf. Financial Cryptography Data Secur.*, 2018, pp. 420–438.
[12] M. I. Mehar et al., "Understanding a revolutionary and flawed grand experiment in blockchain: The dao attack," *J. Cases Inf. Technol.*, vol. 21, no. 1, pp. 19–32, 2019.
[13] P. Voigt and A. Von dem Bussche, "The eu general data protection regulation (GDPR)," *A Practical Guide*, 1st Ed., Berlin, Germany: Springer, vol. 10, no. 3152676, pp. 10–5555, 2017.
[14] J. M. L. Alfonsín, "Argentina: The right to be forgotten," in *The Right To Be Forgotten*. Berlin, Germany: Springer, 2020, pp. 239–248.
[15] G. Ateniese, B. Magri, D. Venturi, and E. Andrade, "Redactable blockchain– or– rewriting history in bitcoin and friends," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2017, pp. 111–126.
[16] G. Ateniese and B. D. Medeiros, "On the key exposure problem in chameleon hashes," in *Proc. Int. Conf. Secur. Commun. Netw.*, 2004, pp. 165–179.
[17] D. Derler, K. Samelin, D. Slamanig, and C. Striecks, "Fine-grained and controlled rewriting in blockchains: Chameleon-hashing gone attribute-based," in *Proc. 26th Annu. Netw. Distrib. Syst. Secur. Symp.*, 2019, pp. 1–15.
[18] Y. Tian, N. Li, Y. Li, P. Szalachowski, and J. Zhou, "Policy-based chameleon hash for blockchain rewriting with black-box accountability," in *Proc. Annu. Comput. Secur. Appl. Conf.*, 2020, pp. 813–828.
[19] G. Panwar, R. Vishwanathan, and S. Misra, "Retrace: Revocable and traceable blockchain rewrites using attribute-based cryptosystems," in *Proc. 26th ACM Symp. Access Control Models Technol.*, 2021, pp. 103–114.
[20] S. Xu, J. Ning, J. Ma, G. Xu, J. Yuan, and R. H. Deng, "Revocable policy-based chameleon hash," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2021, pp. 327–347.
[21] J. Ma, S. Xu, J. Ning, X. Huang, and R. H. Deng, "Redactable blockchain in decentralized setting," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1227–1242, 2022.
[22] S. Xu, J. Ning, J. Ma, X. Huang, and R. H. Deng, "K-time modifiable and epoch-based redactable blockchain," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4507–4520, 2021.
[23] Y. Jia, S.-F. Sun, Y. Zhang, Z. Liu, and D. Gu, "Redactable blockchain supporting supervision and self-management," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2021, pp. 844–858.
[24] G. Ateniese et al., "Provable data possession at untrusted stores," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, 2007, pp. 598–609.
[25] A. Juels and B. S. Kaliski Jr, "PORs: Proofs of retrievability for large files," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, 2007, pp. 584–597.
[26] H. Shacham and B. Waters, "Compact proofs of retrievability," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2008, pp. 90–107.
[27] D. Schröder and H. Schröder, "Verifiable data streaming," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2012, pp. 953–964.
[28] J. Krupp, D. Schröder, M. Simkin, D. Fiore, G. Ateniese, and S. Nürnberger, "Nearly optimal verifiable data streaming," in *Public-Key Cryptography–PKC 2016*. Berlin, Germany: Springer, 2016, pp. 417–445.
[29] J. Wei, G. Tian, J. Shen, X. Chen, and W. Susilo, "Optimal verifiable data streaming protocol with data auditing," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2021, pp. 296–312.
[30] M. Miao, J. Wei, J. Wu, K.-C. Li, and W. Susilo, "Verifiable data streaming with efficient update for intelligent automation systems," *Int. J. Intell. Syst.*, vol. 37, no. 2, pp. 1322–1338, 2022.
[31] I. Puddu, A. Dmitrienko, and S. Capkun, "μchain: How to forget without hard forks," *IACR Cryptol. ePrint Arch. 2017/106*, vol. 2017, 2017, Art. no. 106.
[32] D. Deuber, B. Magri, and S. A. K. Thyagarajan, "Redactable blockchain in the permissionless setting," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 124–138.
[33] J. Camenisch, D. Derler, S. Krenn, H. C. Pöhls, K. Samelin, and D. Slamanig, "Chameleon-hashes with ephemeral trapdoors," in *Proc. IACR Int. Workshop Public Key Cryptography*, 2017, pp. 152–182.
[34] V. Goyal, O. Pandey, A. Sahai, and B. Waters, "Attribute-based encryption for fine-grained access control of encrypted data," in *Proc. 13th ACM Conf. Comput. Commun. Secur.*, 2006, pp. 89–98.
[35] T. Kohno, A. Stubblefield, A. D. Rubin, and D. S. Wallach, "Analysis of an electronic voting system," in *Proc. IEEE Symp. Secur. Privacy*, 2004, pp. 27–40.
[36] X. Chen et al., "Efficient generic on-line/off-line (threshold) signatures without key exposure," *Inf. Sci.*, vol. 178, no. 21, pp. 4192–4203, 2008.
[37] S. Gorbunov, L. Reyzin, H. Wee, and Z. Zhang, "Pointproofs: Aggregating proofs for multiple vector commitments," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 2007–2023.
[38] G. Fuchsbauer, E. Kiltz, and J. Loss, "The algebraic group model and its applications," in *Proc. Annu. Int. Cryptol. Conf.*, 2018, pp. 33–62.

Guohua Tian received the BS degree from the School of Mathematics and Information Science, Shaanxi Normal University, in 2016, and the MS degree from School of Mathematics and Statistics, Xidian University, in 2019. Currently, he is working toward the PhD degree majoring in cyberspace security with the School of Cyber Engineering, Xidian University. His research interests include network and information security, cloud computing, data deduplication and blockchain.

Jianghong Wei received the PhD degree in Information Security from Zhengzhou Information Science and Technology Institute, Zhengzhou, China, in 2016. He is currently a lecturer with the State Key Laboratory of Mathematical Engineering and Advanced Computing, Zhengzhou, China. His research interests include applied cryptography and network security.

Mirosław Kutyłowski received the PhD degree from the University of Wroclaw, in 1986. He is a full professor of computer science with the Wroclaw University of Technology. His research interests include algorithms for distributed and ad hoc systems, privacy protection, security and pragmatic cryptography. he is particularly involved in research on malicious cryptography and the problems that arise due to interplay of modern IT technology and law. He has been an expert in many e-government initiatives serving public institutions and business organizations.

Willy Susilo (Fellow, IEEE) received the PhD degree in computer science from the University of Wollongong, Australia. He is a distinguished professor and head of School of Computing and Information Technology and the director of Institute of Cyber security and Cryptology (iC^2) at the University of Wollongong. He was awarded the prestigious Australian Research Council (ARC) Future Fellow. His main research interests include cryptography and in-formation security. His main contribution is in the area of digital signature schemes. He has served as a program committee member in dozens of international conferences. He has published numerous publications in the area of digital signature schemes and encryption schemes.



Xinyi Huang received the PhD degree from the School of Computer Science and Software Engineering, University of Wollongong, Australia. He is currently a professor with the Artificial Intelligence Thrust, Information Hub, Hong Kong University of Science and Technology (Guangzhou), China. His research interests include applied cryptography and network security. He has authored more than 150 research papers in refereed international conferences and journals.

Xiaofeng Chen (Senior Member, IEEE) received the BS and MS degrees in mathematics from Northwest University, China, in 1998 and 2000, respectively, and the PhD degree in cryptography from Xidian University in 2003. Currently, he works with Xidian University as a professor. His research interests include applied cryptography and cloud computing security. He has published more than 200 research papers in referred international conferences and journals. His work has been cited more than 14000 times at Google Scholar. He is a member of the Editorial Board for *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Knowledge and Data Engineering*, *International Journal of Foundations of Computer Science*, etc. He has served as the program/general chair or program committee member in more than 30 international conferences.